

# Artificial Intelligence

## Final Exam — Complete Revision Notes

Every numerical shows its formula in proper notation, then the working

### Topics covered, in syllabus order:

1. Constraint Satisfaction Problems — constraints, graph, domain shrinking, solved examples
2. Expert Systems — inference, propositional and predicate logic with solved proofs, Prolog
3. NLP — full definitions and descriptions, plus TF-IDF numerical
4. Neural Network — full backpropagation numerical
5. Linear Regression — slope and intercept numerical
6. Clustering — K-Means and SSE numericals
7. Decision Tree — Gini and Entropy numericals
8. Accuracy — confusion matrix, precision, recall, F1

**Exam tip:** In every numerical, the blue 'Formula' line gives the rule in mathematical notation, then the line below substitutes the numbers. KNN and Naive Bayes are not in the syllabus, so they are left out. Forward checking in CSP is excluded as you asked.

# 1. Constraint Satisfaction Problems — CSP

A CSP is a problem where you must give every variable a value, and the values you choose must obey a set of rules. Everyday examples are Sudoku, scheduling meetings with no clashes, and colouring a map so that no two touching regions share a colour.

## The three building blocks

- **Variables** — the things you assign a value to. Example: each region of a map.
- **Domain** — the set of values a variable is allowed to take. Example: a region can be red, green or blue.
- **Constraints** — the rules saying which combinations are allowed. Example: two touching regions must not be the same colour.

A **solution** is an assignment that is **complete**, meaning every variable has a value, and **consistent**, meaning no rule is broken.

## Types of constraint

- **Unary** — affects one variable. Example: region SA must not be green.
- **Binary** — affects two variables. Example: SA must not equal WA.
- **Global or higher order** — affects three or more variables. Example: all values must be different.
- **Preference or soft** — not a hard rule, just a wish with a cost. Example: a morning slot costs 1 point, an afternoon slot costs 2.

## The constraint graph — the 'understand graph' part

A CSP can be drawn as a graph. **Each node is a variable. Each line, called an arc, is a constraint between two variables.** The picture shows which variables affect which. For the exam, be able to draw nodes for the variables and arcs for the rules between them.

## Variable types

- **Discrete, finite domain** — limited choices. With  $n$  variables and domain size  $d$ , there are  $d^n$  possible complete assignments.
- **Discrete, infinite domain** — integers or strings, as in job scheduling.
- **Continuous** — like start and end times. If constraints are linear they can be solved quickly by linear programming.

## How CSPs are solved

- **Backtracking search** — depth first. Assign one variable at a time; if a rule breaks, go back and try another value.
- **Minimum Remaining Value**, also called Most Constrained Variable — choose the variable with the fewest legal values left. The 'fail first' idea: spot a dead end early.
- **Degree heuristic** — choose the variable connected to the most other unassigned variables.
- **Least Constraining Value** — for a chosen variable, try the value that removes the fewest options from its neighbours.

## Domain shrinking with arc consistency

This is the **domain shrinking** item in your syllabus. Delete values that can never work, before searching, so the problem becomes smaller.

- **Node consistency** — apply unary rules first. If SA cannot be green, delete green from SA.
- **Arc consistency** — an arc  $X$  to  $Y$  is consistent if every value left in  $X$  has at least one matching value in  $Y$ . If a value of  $X$  has no partner in  $Y$ , delete it from  $X$ .

### AC-3 algorithm:

- Put all arcs into a queue.
- Take an arc  $X$  to  $Y$ . Remove from  $X$  any value with no compatible value in  $Y$ .
- If you removed a value from  $X$ , add back every arc  $Z$  to  $X$ , because shrinking  $X$  may now affect its other neighbours.
- Repeat until the queue is empty.

#### ■ Worked numerical — domain shrinking with $A < B < C$

**Formula** → Arc consistency: keep value  $v$  in  $X$  only if a matching value in  $Y$  (here  $A < B < C$ )

Variables  $A, B, C$  with  $A < B$  and  $B < C$ . Start: every domain is  $\{1, 2, 3, 4\}$ .

##### Step 1 — make $A$ consistent with $B$ (each $A$ needs a larger $B$ )

Value 4 in  $A$  has no larger  $B$ , so delete 4.  $A = \{1, 2, 3\}$ .

##### Step 2 — make $B$ consistent with $A$ and $C$

Value 1 has no smaller  $A$ ; value 4 has no larger  $C$ .  $B = \{2, 3\}$ .

##### Step 3 — tighten using the new domains

Value 3 in  $A$  has no larger  $B$  left, delete it.  $A = \{1, 2\}$ . Re-check  $C$ :  $C = \{3, 4\}$ .

##### Result

$A = \{1, 2\}$   $B = \{2, 3\}$   $C = \{3, 4\}$ . Domains shrank from 4 values each to 2 each, with no guessing.

#### ■ Second numerical — flight capacity bounds

**Formula** →  $F_{\min} = \text{total} - (\text{other flight's maximum})$

Flights  $F1, F2$  with capacities 165 and 385. Domains:  $F1 \in [0, 165]$ ,  $F2 \in [0, 385]$ . Constraint:  $F1 + F2 = 420$ .

$$F1 \geq 420 - 385 = 35 \text{ so } F1 \quad [35, 165]$$

$$F2 \geq 420 - 165 = 255 \text{ so } F2 \quad [255, 385]$$

## Solved examples of the CSPs, step by step

### ■ Example 1 — Map colouring solved by backtracking

**Formula** → Adjacent regions must differ:  $\text{colour}(A) \neq \text{colour}(B)$

Regions WA, NT, SA, Q, NSW, V, T. Colours red, green, blue. SA touches WA, NT, Q, NSW, V; WA-NT; NT-Q; Q-NSW; NSW-V; T has no neighbours.

**Assign one region at a time, taking the first colour that breaks no rule**

WA = red

NT = green differ from WA red

SA = blue differ from WA red and NT green

Q = red differ from NT green and SA blue

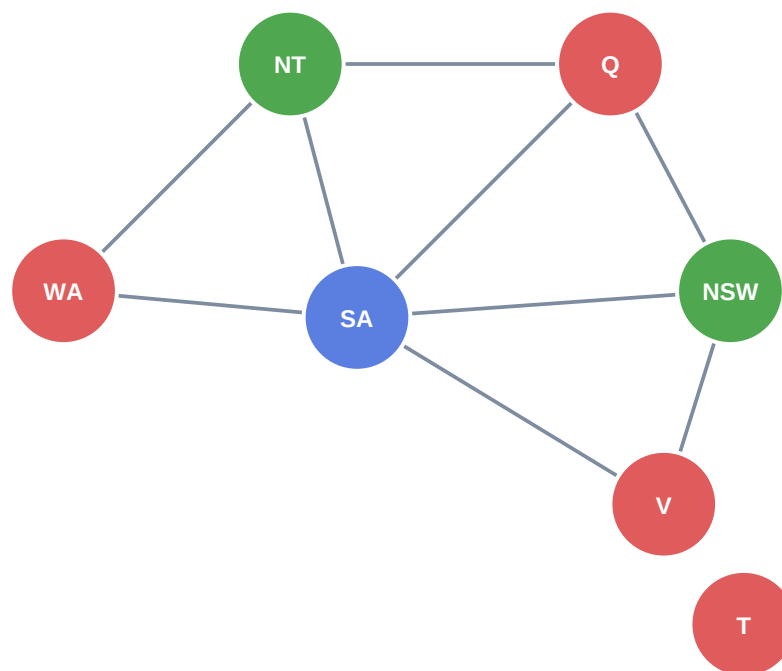
NSW = green differ from Q red and SA blue

V = red differ from NSW green and SA blue

T = red no neighbours, so any colour works

#### **Solution**

WA red, NT green, SA blue, Q red, NSW green, V red, T red. Every neighbouring pair differs, so it is complete and consistent.



Map colouring constraint graph. Each circle is a region and each line is a 'must differ' constraint. The colours show the solution above — every line joins two different colours. T has no line, so it is unconstrained.

### ■ Example 2 — 8 Queens solved by backtracking

**Formula** → Safe if  $\text{row}(i) \neq \text{row}(j)$  and  $|i - j| \neq |\text{row}(i) - \text{row}(j)|$  (not same row or diagonal)

One queen per column, V1 to V8 give the rows, domain rows 1 to 8. Backtracking places a queen per column and backs up at any dead end.

**A complete solution, column = row**

col1=1, col2=5, col3=8, col4=6, col5=3, col6=7, col7=2, col8=4. Row list: 1, 5, 8, 6, 3, 7, 2, 4.

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | Q |   |   |   |   |   |   |
| 2 |   |   |   |   |   | Q |   |
| 3 |   |   |   | Q |   |   |   |
| 4 |   |   |   |   |   |   | Q |
| 5 |   | Q |   |   |   |   |   |
| 6 |   |   |   | Q |   |   |   |
| 7 |   |   |   |   |   | Q |   |
| 8 |   |   | Q |   |   |   |   |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

One valid 8-Queens solution. Columns 1 to 8 along the bottom, rows 1 to 8 down the left. Each column holds one queen, and no two share a row, column, or diagonal.

### ■ Example 3 — Cryptarithmic TWO + TWO = FOUR solved

```

  T W O
+ T W O
-----
F O U R

```

**Formula** → Each column:  $\text{digit} + \text{digit} + \text{carry\_in} = \text{result\_digit} + 10 \times \text{carry\_out}$

Units:  $O + O = R + 10 \times X_1$

Tens:  $X_1 + W + W = U + 10 \times X_2$

Hundreds:  $X_2 + T + T = O + 10 \times X_3$

Thousands:  $X_3 = F$ ,  $T \neq 0$ ,  $F \neq 0$ , all digits different

**Try  $T = 8$ ,  $W = 3$ ,  $O = 6$**

Units:  $6 + 6 = 12 \rightarrow R = 2$ , carry  $X_1 = 1$ .

Tens:  $1 + 3 + 3 = 7 \rightarrow U = 7$ , carry  $X_2 = 0$ .

Hundreds:  $0 + 8 + 8 = 16 \rightarrow O = 6$ , carry  $X_3 = 1$ .

Thousands:  $F = X_3 = 1$ .

**Answer**

$836 + 836 = 1672$ . Digits T8 W3 O6 F1 U7 R2 all different.

### ■ Example 4 — Cryptarithmic SEND + MORE = MONEY solved

```

  S E N D
+ M O R E
-----
M O N E Y

```

**Formula** → Each column:  $\text{digit} + \text{digit} + \text{carry\_in} = \text{result\_digit} + 10 \times \text{carry\_out}$ ; all letters different

Two 4-digit numbers give at most a 5-digit number starting with 1, so  $M = 1$ .

For that leading 1,  $S + M + \text{carry}$  must reach 10, forcing  $S = 9$  and  $O = 0$ .

The remaining columns give  $E = 5$ ,  $N = 6$ ,  $R = 8$ ,  $D = 7$ ,  $Y = 2$ .

**Answer**

$9567 + 1085 = 10652$ .

### ■ Example 5 — Sudoku, how a cell is solved

**Formula** → Each digit 1-9 appears once per row, once per column, once per 3×3 box (all-different)

Pick an empty cell. Cross out every digit already in its row, its column, or its 3×3 box.

If only one digit survives, it must go in the cell — this is domain shrinking in Sudoku. Where more than one digit remains, guess and backtrack if it fails.

**Exam tip:** Likely 10 mark question: define the three building blocks, draw the constraint graph for map colouring, then show domain shrinking or solve map colouring by backtracking.

## 2. Expert Systems

An expert system is an AI program that copies the decision making of a human expert in one specific field. It uses stored knowledge plus rules to solve problems, give advice, or make decisions the way a specialist would.

### Uses and famous examples

- Helps a human expert combine many decisions, or replaces an expert when none is available.
- **DENDRAL** identifies chemical structures, first used 1965 at Stanford.
- **MYCIN** diagnoses blood disorders, first used 1979.
- **PROSPECTOR** finds mining sites; **PUFF** diagnoses respiratory conditions.
- **XCON** configured computers and saved the company DEC about 70 million dollars a year, first used 1984.

### When to use one, and its weaknesses

- Use when the payoff is high, the task is knowledge heavy, a human expert exists, it is not language heavy, many test cases exist, and no creativity or physical skill is needed.
- Weaknesses: no common sense, no creativity, no automatic learning, no sensory experience, and poor at knowing when no answer exists.

### Two main components

- **Knowledge base** — the store of facts and rules about the field.
- **Inference engine** — applies the rules to the known facts to work out new facts, and can explain its reasoning.

### Knowledge representation — how knowledge is stored

- **Rules** — if-then statements, also called production rules. The if part is the premise, the then part is the action. Example: if temperature is below 32 degrees Fahrenheit, then water freezes.
- **Frames** — a structure holding an object with its properties, called slots, and their values.
- **Semantic networks** — knowledge drawn as linked concepts.
- **Propositional and predicate logic** — covered below.

**Declarative knowledge** states facts as logic and is easy to change. **Procedural knowledge** describes how to do a task; it is easy to use but hard to change.

### Reasoning — this is inference

- **Deduction** — general to specific. All men are mortal; Shakespeare is a man; therefore Shakespeare is mortal.
- **Induction** — specific to general. Shakespeare, Newton and Einstein are mortal; therefore all men are mortal.

### Propositional Logic

A **proposition** is a simple statement that is either true or false, for example 'Fish is expensive'. It always has a truth value. Propositions combine using operators.

Operators: **AND** true only when both parts are true; **OR** true when at least one is true; **NOT** flips the value; **IMPLIES** means if-then; **EQUIVALENT** true when both sides match.

| P | Q | P AND Q | P OR Q | NOT P |
|---|---|---------|--------|-------|
| F | F | F       | F      | T     |
| F | T | F       | T      | T     |

|   |   |   |   |   |
|---|---|---|---|---|
| T | F | F | T | F |
| T | T | T | T | F |

**Implies**, written A implies B, is false in only one case: when A is true but B is false. Otherwise it is true.

| A | B | A implies B |
|---|---|-------------|
| F | F | T           |
| F | T | T           |
| T | F | F           |
| T | T | T           |

**Logic laws to know:** Idempotent, Commutative, Distributive, Associative, Absorptive, and De Morgan.

De Morgan:  $\text{NOT}(A \text{ AND } B) = \text{NOT } A \text{ OR } \text{NOT } B$ , and  $\text{NOT}(A \text{ OR } B) = \text{NOT } A \text{ AND } \text{NOT } B$ .

**Limitations:** propositional logic treats whole sentences as single symbols, cannot talk about specific objects, and is hard to update. That is why we use first order logic.

### ■ Propositional logic — sentence to logic and logic to sentence

Let P = it is raining, Q = I take an umbrella, R = I get wet.

#### Sentence to logic

If it is raining then I take an umbrella  $\rightarrow P \rightarrow Q$

It is raining and I do not take an umbrella  $\rightarrow P \wedge \neg Q$

If it rains and I have no umbrella, then I get wet  $\rightarrow (P \wedge \neg Q) \rightarrow R$

#### Logic to sentence

$P \rightarrow Q \rightarrow$  If it is raining, then I take an umbrella

$\neg Q \rightarrow \neg P \rightarrow$  If I do not take an umbrella, then it is not raining

$P \vee Q \rightarrow$  Either it is raining or I take an umbrella

### ■ Propositional logic — prove a conclusion from rules

**Formula**  $\rightarrow$  Modus Ponens:  $(X \rightarrow Y) \text{ and } X \rightarrow Y$  (  $\rightarrow$  = and,  $\neg$  = not,  $\therefore$  = therefore)

Let S = it snows, DM = I study, A = I get an A.

#### Given

Rule 1:  $S \rightarrow DM$  Rule 2:  $DM \rightarrow A$  Fact: S is true

#### Goal: prove A

Step 1: S true and  $(S \rightarrow DM) \therefore DM$  true.

Step 2: DM true and  $(DM \rightarrow A) \therefore A$  true.

#### Proved

A is true. Chaining gives  $S \rightarrow A$  (hypothetical syllogism).

## First Order Logic — predicate logic

First Order Logic, also called predicate logic or predicate calculus, can talk about objects, their relationships, and general statements.

- **Constants** — single objects such as Marc, Elise, Bicycle.
- **Predicates** — relationships or properties. Person(Marc) means Marc is a person; Knows(Marc, Elise) means they know each other.
- **Functions** — indirect references such as fatherOf, oneMoreThan.
- **Atomic sentence** — a single predicate statement, like Person(Marc).
- **Compound sentence** — atomic sentences joined by AND, OR, NOT, implies, or equivalence.



- **Variables** — stand in for unknown objects, like x.
- **Universal quantifier**  $\forall$  — 'for all'; holds for everything substituted.
- **Existential quantifier**  $\exists$  — 'there exists': at least one thing makes it true.

**Exam tip:** Predicates are written with their objects in brackets, like `Person(Marc)` or `Knows(Marc, Elise)`. Symbols:  $\forall$  for all,  $\exists$  there exists,  $\wedge$  and,  $\vee$  or,  $\neg$  not,  $\rightarrow$  implies.

## ■ Predicate logic — sentence to logic and logic to sentence

### Sentence to logic

Marc is a person  $\rightarrow$  `Person(Marc)`

Marc rides a bicycle  $\rightarrow$  `Rides(Marc, Bicycle)`

Marc knows Elise and Elise knows Marc  $\rightarrow$  `Knows(Marc, Elise)  $\wedge$  Knows(Elise, Marc)`

If Marc knows Elise then Elise knows Marc  $\rightarrow$  `Knows(Marc, Elise)  $\rightarrow$  Knows(Elise, Marc)`

Everyone who knows Elise is a person  $\rightarrow \forall x$  `Knows(x, Elise)  $\rightarrow$  Person(x)`

Someone knows Marc  $\rightarrow \exists x$  `Knows(x, Marc)`

### Logic to sentence

`Rides(Marc, Scooter)  $\vee$  Rides(Marc, Bicycle)`  $\rightarrow$  Marc rides a scooter or a bicycle

`Knows(Marc, Elise)  $\wedge$   $\neg$ Knows(Elise, Marc)`  $\rightarrow$  Marc knows Elise, but Elise does not know Marc

$\forall x$  `Person(x)  $\rightarrow$  Mortal(x)`  $\rightarrow$  Every person is mortal

## ■ Predicate logic — prove a goal from facts and rules

**Formula**  $\rightarrow$  If a rule's if-part matches known facts, its then-part becomes a new fact (Modus Ponens).

### Facts

`Father(John, Jim), Father(Jack, John), Mother(Jane, Jim)`

### Rules

R1: `Father(a, b)  $\rightarrow$  Parent(a, b)`

R2: `Mother(a, b)  $\rightarrow$  Parent(a, b)`

R3: `Parent(a, c)  $\wedge$  Parent(c, b)  $\rightarrow$  Grandparent(a, b)`

### Goal: prove `Grandparent(Jack, Jim)`

Step 1: `Father(Jack, John)` and R1  $\therefore$  `Parent(Jack, John)`.

Step 2: `Father(John, Jim)` and R1  $\therefore$  `Parent(John, Jim)`.

Step 3: `Parent(Jack, John)  $\wedge$  Parent(John, Jim)` and R3 ( $a=Jack, c=John, b=Jim$ )  $\therefore$  `Grandparent(Jack, Jim)`.

### Proved

`Grandparent(Jack, Jim)` is true, so Jack is the grandparent of Jim.

## ■ Predicate logic — the classic Socrates style proof

**Formula**  $\rightarrow$  Modus Ponens:  $(X \rightarrow Y)$  and  $X \quad Y$

### Given

Fact: `Man(Shakespeare)` Rule:  $\forall x$  `Man(x)  $\rightarrow$  Mortal(x)`

### Goal: prove `Mortal(Shakespeare)`

Step 1: put  $x = \text{Shakespeare}$  into the rule  $\rightarrow$  `Man(Shakespeare)  $\rightarrow$  Mortal(Shakespeare)`.

Step 2: `Man(Shakespeare)` is a fact.

Step 3: by Modus Ponens  $\therefore$  `Mortal(Shakespeare)`.

### Proved

`Mortal(Shakespeare)` is true. This is deduction, general rule to specific case.

## Inference methods — forward and backward chaining

- **Forward chaining — data driven.** Start from known facts and keep applying rules to make new facts; the top matching rule fires once each cycle and adds a fact; it stops when no rule can fire. It may fire rules unrelated to the goal, so it is inefficient when you only want one fact.
- **Backward chaining — goal driven.** Start from a goal; find a rule whose then-part is the goal; if its if-part matches facts, the goal is proved; otherwise the if-part becomes a subgoal and the search repeats.
- **Choosing:** gather then conclude → forward; start with a hypothesis then hunt for facts → backward.

**Exam tip:** Likely 10 mark question: define an expert system, draw its two components, and explain inference using predicate logic or forward versus backward chaining.

# Prolog — Programming in Logic

Prolog is the practical use of First Order Logic. It is a **declarative** language: you describe what is true. A Prolog program is a database of **facts** and **rules**, and you run it by asking **queries**.

**Exam tip:** Prolog uses round brackets as part of its real syntax, so the code below shows them. You must recognise the real syntax in the exam.

## The pieces of a Prolog program

- **Fact** — something always true. Head, no body.
- **Rule** — true only if its body is true. Head, then :- meaning 'if', then a body.
- **Query** — a question, starting with ?-
- **Predicate** — the name before the brackets, such as father or parent.
- **Arguments** — inside the brackets. A **constant** starts lower case, like jane; a **variable** starts upper case, like X.
- **Clause** — a head with an optional body. In a body, a comma means AND and a semicolon means OR. Every clause ends with a full stop.

## Example database

```
% Facts
father(john, jim).
father(jack, john).
mother(jane, jim).

% Rules
parent(X, Y) :- father(X, Y).
parent(X, Y) :- mother(X, Y).
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

In words: John is the father of Jim. Jack is the father of John. Jane is the mother of Jim. X is a parent of Y if X is the father or mother of Y. X is a grandparent of Y if X is a parent of some Z and that Z is a parent of Y.

## Asking questions

```
?- father(john, jim).
true

?- mother(jane, fred).
false

?- father(X, jim).
X = john

?- grandparent(jack, jim).
true
```

### ■ How Prolog answers `grandparent(jack, jim)`, step by step

Goal: `grandparent(jack, jim)` is true if `parent(jack, Z)` and `parent(Z, jim)`, for some Z.

#### Step 1 — solve `parent(jack, Z)`

Try `father(jack, Z)`. The fact `father(jack, john)` matches, so `Z = john`.

#### Step 2 — solve `parent(john, jim)`

Try `father(john, jim)`. The fact matches, so this is true.

#### Step 3 — both parts true

Both conditions succeed, so `grandparent(jack, jim)` is true.

**Underneath:** Prolog uses **unification** (matching a goal to a fact or rule head and filling in variables) and **backtracking** (if a match fails, go back and try another).

## 3. Natural Language Processing — NLP

**Definition.** NLP is a branch of AI that helps computers understand, interpret and respond to human language, the way we speak or write. It lets a device read your words, work out what you mean, and reply sensibly. It combines **linguistics** (how language works) with **machine learning** (learning from data).

**Why it matters.** NLP powers smart assistants, translators, chatbots and auto email replies. It lets a computer break sentences into parts, understand meaning, detect emotion, translate, and summarise.

### Text mining versus information retrieval

- **Text mining** — pulling useful patterns and knowledge out of unstructured text, such as reading thousands of reviews to find common complaints. Techniques: NER, sentiment analysis, topic modelling, classification, keyword extraction.
- **Information retrieval** — finding the relevant documents from a large collection for a query, such as a Google search. Techniques: search engines, ranking like TF-IDF and BM25, query expansion.
- **Memory aid:** information retrieval is finding the right book; text mining is reading it to learn what it says.

### Core tasks of NLP

- **Tokenization** — breaking text into words or sentences. 'NLP is fun' → NLP, is, fun.
- **Part of Speech tagging** — labelling each word noun, verb, adjective. 'Dogs bark loudly' → noun, verb, adverb.
- **Named Entity Recognition** — classifying real things. 'Elon Musk founded SpaceX in 2002' → person, organisation, date.
- **Dependency parsing** — how words relate, such as subject and object. Helps grammar analysis and question answering.
- **Sentiment analysis** — detecting tone. 'I love this product' → positive.

### Subfields of NLP

- **Sentiment analysis** — emotions or opinions as positive, negative or neutral.
- **Translation** — one language to another. Hello → Bonjour.
- **Speech recognition** — spoken language to text. Siri, Alexa.
- **Question answering** — read a passage and answer questions. Chatbots, AI tutors.
- **Text summarization** — a shorter summary. **Extractive** pulls key sentences; **abstractive** rewrites in fewer words.
- **Text generation** — creating new text. ChatGPT, story generation, auto replies.

### Ambiguity — the main challenge

- **Lexical** — one word, many meanings. 'Root'; 'design' (noun or verb).
- **Syntactic** — structure read more than one way. 'A man saw a boy with a telescope'.
- **Semantic** — meaning unclear. 'The bike hit the pole while it was moving'.
- **Pragmatic** — depends on context. 'The police is coming'.

### Approaches to NLP

- **Rule based** — hand written rules and patterns. Simple, no training data. Example: 'not' + positive word → negative. Rigid and limited.

- **Statistical NLP** — probabilities from large text. N-grams predict the next word; Hidden Markov Models do tagging and speech. Needs data, weak on long range context.
- **Classical machine learning** — features like TF-IDF or n-grams into SVM or Logistic Regression. More accurate; needs manual feature engineering.
- **Deep learning** — neural networks. Word embeddings turn words into vectors (king – man + woman ≈ queen). RNNs and LSTMs read word by word. Transformers (BERT, GPT) read all words at once. Most powerful; learns context automatically.

## Preprocessing in NLP

Raw text → Clean → Tokenize → Normalize → Remove stop words → Lemmatize / Stem → Vectorize

- **Clean** — remove punctuation and symbols. **Tokenize** — split into words. **Normalize** — lower case.
- **Remove stop words** — drop common words like the, is, and.
- **Stemming** chops endings to a rough root (studies → studi). **Lemmatization** gives the dictionary base form (studies → study).
- **Vectorize** — turn words into numbers (Bag of Words or TF-IDF).

## Text representation

- **Bag of Words** — count how many times each word appears, ignoring grammar and order.
- **TF-IDF** — lowers the weight of common words and raises rare, important ones.

## TF-IDF in detail

TF = (count of the term in the document) / (total terms in the document)  
 IDF =  $\log( N / df )$  df = number of documents that contain the term  
 TF-IDF = TF × IDF

N is the total number of documents. A high TF-IDF means the word is frequent here but rare overall, so it is a strong, distinctive clue.

**Exam tip:** Your syllabus note 'do not log for small values' means: when  $N / df$  is small, the lecturer skips the log and uses  $N / df$  directly. The query terms are chosen freely; IDF is computed for all words in advance and only the query terms are used in the score.

### ■ Numerical A — standard TF-IDF with log

A document has 100 words; 'cat' appears 3 times. There are 10,000,000 documents and 'cat' is in 1,000.

**Formula** →  $TF = (\text{count}) / (\text{total terms})$

$TF = 3 / 100 = 0.03$

**Formula** →  $IDF = \log( N / df )$

$IDF = \log( 10,000,000 / 1,000 ) = \log( 10,000 ) = 4$

**Formula** →  $TF-IDF = TF \times IDF$

$\text{score} = 0.03 \times 4 = 0.12$

### ■ Numerical B — ranking 5 documents

**Formula** →  $\text{Relative TF} = (\text{count of term}) / (\text{total words in the document})$

Five documents D1 to D5. Relative TF table:

| Doc | Nato   | Cricket | Shareef | Missing | Court  |
|-----|--------|---------|---------|---------|--------|
| D1  | 0.5    | 0       | 0.375   | 0.125   | 0      |
| D2  | 0.1111 | 0       | 0.1111  | 0.4444  | 0.3333 |

|    |        |       |        |        |        |
|----|--------|-------|--------|--------|--------|
| D3 | 0      | 0.625 | 0.25   | 0      | 0.125  |
| D4 | 0.4285 | 0     | 0.5714 | 0      | 0      |
| D5 | 0.1111 | 0     | 0.2222 | 0.3333 | 0.3333 |

### ■ Step 2 — IDF for each term

**Formula** →  $IDF = N / df$  with  $N = 5$  (log skipped, small values)

Nato D1,D2,D4,D5 →  $df=4$  →  $IDF = 5/4 = 1.25$

Cricket D3 →  $df=1$  →  $IDF = 5/1 = 5$

Shareef all 5 →  $df=5$  →  $IDF = 5/5 = 1$

Missing D1,D2,D5 →  $df=3$  →  $IDF = 5/3 = 1.667$

Court D2,D3,D5 →  $df=3$  →  $IDF = 5/3 = 1.667$

### ■ Step 3 — score for the chosen query 'shareef + court'

**Formula** →  $score(doc) = \sum \text{over query terms of } (TF \times IDF)$

Shareef  $IDF = 1$ , Court  $IDF = 1.667$ .

D1:  $(0.375 \times 1) + (0 \times 1.667) = 0.375 + 0 = 0.375$

D2:  $(0.1111 \times 1) + (0.3333 \times 1.667) = 0.111 + 0.555 = 0.666$

D3:  $(0.25 \times 1) + (0.125 \times 1.667) = 0.25 + 0.208 = 0.458$

D4:  $(0.5714 \times 1) + (0 \times 1.667) = 0.571 + 0 = 0.571$

D5:  $(0.2222 \times 1) + (0.3333 \times 1.667) = 0.222 + 0.555 = 0.777$

#### Ranking (highest first)

D5 = 0.777 (1), D2 = 0.666 (2), D4 = 0.571 (3), D3 = 0.458 (4), D1 = 0.375 (5).

## Sentiment analysis and extra-semantic information

**Extra-semantic information** means the hidden meaning or feeling behind how language is used. 'Customer is unhappy with their service' carries discontent. Sentiment covers emotions (fear, love, hate), feelings (warmth, excitement), mood, and one day even sarcasm and lies.

## Applications of text analytics

- Document classification and email filtering (spam detection).
- Sentiment analysis of Twitter posts and reviews.
- Visualization such as word clouds; document clustering.

## Modern NLP — Transformers and GPT

- **Transformer** — a deep learning design from 2017. Unlike RNNs it processes a whole sentence at once using **self-attention**, understanding context in parallel.
- **GPT** = Generative Pre-trained Transformer. **Generative**: produces text. **Pre-trained**: already learned from huge text. **Transformer**: its architecture.
- **BERT** reads context both directions; **GPT** reads left to right.

## Challenges in NLP

- Ambiguity in syntax and semantics; sarcasm and idioms; low resource languages; bias in data; high computational cost.

## 4. Neural Network — with full numerical

A neural network is a model loosely based on the brain. Neurons (nodes) sit in layers; each connection has a weight. A neuron multiplies its inputs by their weights, adds a bias, then passes the total through an activation function.

### Structure and key ideas

- **Input layer** takes features; **hidden layers** process them; **output layer** gives the answer. More hidden layers means deeper (deep learning).
- **Weight** is connection strength; learning adjusts weights. **Bias** shifts the activation, the same  $b$  as in  $y = mx + b$ .
- **Activation** (often sigmoid) squashes the value into 0 to 1, an S-shaped curve.
- **Backpropagation** runs the input forward, compares with the target, then pushes the error backward, adjusting weights.

### The formulas you need

$$z = \Sigma(x_i \times w_i) + b \quad a = g(z) = 1 / (1 + e^{-z})$$

$$\delta_{\text{output}} = (T - O) \times O \times (1 - O)$$
$$\delta_{\text{hidden}} = O \times (1 - O) \times \Sigma(\delta_{\text{next}} \times w)$$

$$w_{\text{new}} = w_{\text{old}} + \eta \times \text{input} \times \delta$$

$g = \text{sigmoid}$ ,  $\eta = \text{learning rate}$ ,  $T = \text{target}$ ,  $O = \text{output}$ ,  $\delta = \text{error}$

#### ■ Main numerical — one full backpropagation pass

Inputs  $x_1, x_2$  feed hidden nodes 3, 4; nodes 3, 4 feed output node 5. All weights 0.1 except  $w_{45} = 1$ .  $\eta = 1$ .  
Example:  $x_1 = 1, x_2 = 1, T = 1$ .

##### Step 1 — forward pass

$$\text{Formula} \rightarrow z = \Sigma(x \times w) + b, \quad a = g(z) = 1 / (1 + e^{-z})$$

$$z_3 = (1 \times 0.1) + (1 \times 0.1) = 0.2 \quad a_3 = g(0.2) = 0.550$$

$$z_4 = (1 \times 0.1) + (1 \times 0.1) = 0.2 \quad a_4 = g(0.2) = 0.550$$

$$z_5 = (0.550 \times 0.1) + (0.550 \times 1) = 0.605 \quad a_5 = g(0.605) = 0.647$$

##### Step 2 — errors (backward)

$$\text{Formula} \rightarrow \delta_{\text{out}} = (T - O) \times O \times (1 - O)$$

$$\delta_5 = (1 - 0.647) \times 0.647 \times (1 - 0.647) = 0.08$$

$$\text{Formula} \rightarrow \delta_{\text{hidden}} = O \times (1 - O) \times \delta_5 \times w$$

$$\delta_3 = 0.550 \times 0.450 \times 0.08 \times 0.1 = 0.002$$

$$\delta_4 = 0.550 \times 0.450 \times 0.08 \times 1 = 0.02$$

##### Step 3 — update weights

$$\text{Formula} \rightarrow w_{\text{new}} = w_{\text{old}} + \eta \times \text{input} \times \delta$$

$$w_{35}' = 0.1 + (1 \times 0.550 \times 0.08) = 0.145$$

$$w_{45}' = 1 + (1 \times 0.550 \times 0.08) = 1.045$$

$$w_{13}' = 0.1 + (1 \times 1 \times 0.002) = 0.102$$

$$w_{23}' = 0.1 + (1 \times 1 \times 0.002) = 0.102$$

$$w_{14}' = 0.1 + (1 \times 1 \times 0.02) = 0.12$$

$$w_{24}' = 0.1 + (1 \times 1 \times 0.02) = 0.12$$

##### Step 4 — check

New forward pass gives  $a_5 = 0.6594$ , so error  $1 - 0.6594 = 0.3406$  is lower than the old 0.353. It learned.



### ■ Second numerical — three input neuron

Inputs 1.0, 0.4, 0.7. Hidden nodes i, j feed output k.  $T = 0.65$ .

$w_{1i}=0.20$ ,  $w_{2i}=0.30$ ,  $w_{3i}=-0.10$  ;  $w_{1j}=0.10$ ,  $w_{2j}=-0.10$ ,  $w_{3j}=0.20$  ;  $w_{ik}=0.10$ ,  $w_{jk}=0.50$ .

#### Forward pass

Formula  $\rightarrow \text{input} = \sum(x \times w)$ ,  $O = g(\text{input})$

i:  $(0.20 \times 1.0) + (0.30 \times 0.4) - (0.10 \times 0.7) = 0.25$   $O_i = g(0.25) = 0.562$

j:  $(0.10 \times 1.0) - (0.10 \times 0.4) + (0.20 \times 0.7) = 0.20$   $O_j = g(0.20) = 0.550$

k:  $(0.10 \times 0.562) + (0.50 \times 0.550) = 0.331$   $O_k = g(0.331) = 0.582$

#### Errors

Formula  $\rightarrow \delta_{\text{out}} = (T - O) \times O \times (1 - O)$

$\delta_k = (0.65 - 0.582) \times 0.582 \times (1 - 0.582) = 0.0165$

Formula  $\rightarrow \delta_{\text{hidden}} = O \times (1 - O) \times \delta_k \times w$

$\delta_i = 0.0165 \times 0.10 \times 0.562 \times 0.438 = 0.0004$

$\delta_j = 0.0165 \times 0.50 \times 0.550 \times 0.450 = 0.0020$

#### Update rule

Formula  $\rightarrow w_{\text{new}} = w_{\text{old}} + \eta \times \delta \times \text{input}$

## 5. Linear Regression — slope and intercept

Linear regression fits a straight line through data to predict a continuous output. Classification predicts a category instead; a real number means regression, a class means classification.

$$y = m \cdot x + b \quad m = \text{slope}, \quad b = \text{intercept}$$

$$m = \frac{n \cdot \sum xy - \sum x \cdot \sum y}{n \cdot \sum x^2 - (\sum x)^2} \quad b = \frac{\sum y - m \cdot \sum x}{n}$$

### ■ Numerical — find the best fit line

Data:  $x = 1, 2, 3, 4$  and  $y = 2, 4, 5, 4$ .  $n = 4$ .

#### Step 1 — the sums

Formula → need:  $\sum x$ ,  $\sum y$ ,  $\sum xy$ ,  $\sum x^2$

$$\sum x = 1+2+3+4 = 10 \quad \sum y = 2+4+5+4 = 15$$

$$\sum xy = (1 \times 2) + (2 \times 4) + (3 \times 5) + (4 \times 4) = 41$$

$$\sum x^2 = 1^2 + 2^2 + 3^2 + 4^2 = 30$$

#### Step 2 — slope

Formula →  $m = (n \cdot \sum xy - \sum x \cdot \sum y) / (n \cdot \sum x^2 - (\sum x)^2)$

$$m = (4 \times 41 - 10 \times 15) / (4 \times 30 - 10^2) = (164 - 150) / (120 - 100) = 14 / 20 = 0.7$$

#### Step 3 — intercept

Formula →  $b = (\sum y - m \cdot \sum x) / n$

$$b = (15 - 0.7 \times 10) / 4 = 8 / 4 = 2.0$$

#### Final line

$$y = 0.7x + 2. \text{ Predict } x = 3: y = 0.7 \times 3 + 2 = 4.1$$

## Measuring the error

### ■ Numerical — checking a fitted line $y = 23.6 + 3.5x$

$x$  = years of experience,  $y$  = salary in thousands.

#### One row, $x = 3$ , actual $y = 30$

Formula →  $\hat{y} = 23.6 + 3.5x$  ( $\hat{y}$  = predicted value)

$$\hat{y} = 23.6 + 3.5 \times 3 = 34.1$$

Formula → error =  $y - \hat{y}$ , squared error =  $(y - \hat{y})^2$

$$|30 - 34.1| = 4.1 \quad (30 - 34.1)^2 = 16.81$$

#### Over all rows

Formula →  $MAE = (1/n) \sum |y - \hat{y}|$   $RMSE = \sqrt{(1/n) \sum (y - \hat{y})^2}$

$$MAE = 4.85 \quad RMSE = 5.37$$

Lower values mean a better fit.

## 6. Clustering — K-Means

Clustering is **unsupervised** learning: no target label. The goal is to group similar points so that points inside a cluster are alike and clusters are different from each other.

### The K-Means algorithm

- Choose K, the number of clusters.
- Pick K starting centroids.
- Assign every point to its nearest centroid (Euclidean distance).
- Recompute each centroid as the mean of its points.
- Repeat until the centroids stop changing.

#### ■ Numerical — K-Means with K = 2, first iteration

Points: P1(1, 1.5), P2(1, 4.5), P3(2, 1.5), P4(2, 3.5), P5(3, 2.5), P6(3, 6.0). Initial centroids C1 = P1(1, 1.5), C2 = P3(2, 1.5).

##### Step 1 — distance to each centroid, assign to nearer

**Formula** →  $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

P2: C1  $\sqrt{(0^2+3^2)}=3.00$  ; C2  $\sqrt{(1^2+3^2)}=3.16$  → C1

P4: C1  $\sqrt{(1^2+2^2)}=2.24$  ; C2  $\sqrt{(0^2+2^2)}=2.00$  → C2

P5: C1  $\sqrt{(2^2+1^2)}=2.24$  ; C2  $\sqrt{(1^2+1^2)}=1.41$  → C2

P6: C1  $\sqrt{(2^2+4.5^2)}=4.92$  ; C2  $\sqrt{(1^2+4.5^2)}=4.61$  → C2

##### Step 2 — clusters

C1 = {P1, P2} C2 = {P3, P4, P5, P6}

##### Step 3 — new centroids

**Formula** → centroid = (  $\Sigma x / n$  ,  $\Sigma y / n$  )

C1 = ( (1+1)/2 , (1.5+4.5)/2 ) = (1, 3)

C2 = ( (2+2+3+3)/4 , (1.5+3.5+2.5+6)/4 ) = (2.5, 3.375)

#### ■ Continuing — second iteration

**Formula** → centroid = (  $\Sigma x / n$  ,  $\Sigma y / n$  )

With C1(1, 3) and C2(2.5, 3.375), reassign: C1 = {P1, P2, P3}, C2 = {P4, P5, P6}.

C1 = ( (1+1+2)/3 , (1.5+4.5+1.5)/3 ) = (1.33, 2.5)

C2 = ( (2+3+3)/3 , (3.5+2.5+6)/3 ) = (2.67, 4.0)

Repeat until the centroids stop moving.

### Judging the clustering — SSE and the elbow method

**SSE (Sum of Squared Error)** measures how tight the clusters are: for every point, take the squared distance to its own centroid, then add them all up. Lower SSE means tighter, better clusters.

### ■ Numerical — SSE for the first-iteration clusters

**Formula** →  $SSE = \sum \text{over all points of } (x - c_x)^2 + (y - c_y)^2$

C1(1, 3) holds P1, P2; C2(2.5, 3.375) holds P3, P4, P5, P6.

$$P1: (1-1)^2 + (1.5-3)^2 = 0 + 2.25 = 2.25$$

$$P2: (1-1)^2 + (4.5-3)^2 = 0 + 2.25 = 2.25$$

$$P3: (2-2.5)^2 + (1.5-3.375)^2 = 0.25 + 3.52 = 3.77$$

$$P4: (2-2.5)^2 + (3.5-3.375)^2 = 0.25 + 0.02 = 0.27$$

$$P5: (3-2.5)^2 + (2.5-3.375)^2 = 0.25 + 0.77 = 1.02$$

$$P6: (3-2.5)^2 + (6-3.375)^2 = 0.25 + 6.89 = 7.14$$

#### Add them all

$$SSE = 2.25 + 2.25 + 3.77 + 0.27 + 1.02 + 7.14 = 16.70$$

As the centroids settle over more iterations, SSE keeps dropping.

**Elbow method:** run K-Means for several K and plot SSE against K. SSE always falls as K grows, but at some point it slows sharply, making an 'elbow'. That K is the best choice.

- **Strengths:** fast, scalable, simple, good for round clusters of similar size.
- **Limitations:** must choose K in advance, depends on the starting centroids, sensitive to outliers, struggles with odd shapes or differing sizes and densities.

## 7. Decision Tree — Gini and Entropy

A decision tree splits data with if-then questions, breaking a mixed group into purer groups. Follow the branches that match an example to a leaf, which gives the class. We choose each split using a purity measure.

### Gini and Entropy — the formulas

$$\text{Gini} = 1 - \sum p_i^2 \quad \text{Entropy} = - \sum p_i \cdot \log_2(p_i)$$

$$\text{Weighted(attribute)} = \sum (n_{\text{branch}} / n) \times \text{value(branch)}$$

$$\text{Information Gain} = \text{Entropy(parent)} - \text{Weighted Entropy(children)}$$

**Rule:** Gini → pick the **lowest** weighted Gini. Entropy → pick the **highest** Information Gain. Pure (all one class) has value 0.

### ■ Gini numerical — should a customer buy a computer

**Formula** →  $\text{Gini} = 1 - \sum p_i^2$  ;  $\text{Weighted} = \sum (n_{\text{branch}}/n) \times \text{Gini}_{\text{branch}}$  ; pick lowest

14 records: 9 Yes, 5 No.

#### Age (three branches)

$$\leq 30: 2Y \ 3N \rightarrow 1 - (2/5)^2 - (3/5)^2 = 1 - 0.16 - 0.36 = 0.48$$

$$31-40: 4Y \ 0N \rightarrow 1 - (4/4)^2 = 0 \text{ (pure)}$$

$$> 40: 3Y \ 2N \rightarrow 1 - (3/5)^2 - (2/5)^2 = 0.48$$

$$\text{Weighted} = (5/14) \times 0.48 + (4/14) \times 0 + (5/14) \times 0.48 = 0.342$$

#### Other attributes

$$\text{Income} = 0.44 \quad \text{Student} = 0.366 \quad \text{Credit} = 0.428$$

#### Pick the root

Age has the lowest Gini (0.342), so Age is the root.

## ■ Continuing the tree below Age

### Inside Age $\leq 30$ (2 Yes, 3 No)

Split on Student: Yes  $\rightarrow$  both buy (Gini 0); No  $\rightarrow$  none buy (Gini 0). Weighted Gini = 0, pure.

### Inside Age $> 40$ (3 Yes, 2 No)

Split on Credit: Fair  $\rightarrow$  all buy (Gini 0); Excellent  $\rightarrow$  none buy (Gini 0). Weighted Gini = 0.

### Final rules

Age 31-40  $\rightarrow$  Yes. Age  $\leq 30$ : Student Yes  $\rightarrow$  Yes, Student No  $\rightarrow$  No. Age  $> 40$ : Credit Fair  $\rightarrow$  Yes, Excellent  $\rightarrow$  No.

## ■ Entropy and Information Gain numerical

**Formula**  $\rightarrow$  Entropy =  $-\sum p \cdot \log_2(p)$  ; Gain = Entropy(whole) - Weighted Entropy ; pick highest

8 records: 3 positive, 5 negative. Choose between Hair, Eyes, Height.

### Whole set

$$S = -(3/8)\log_2(3/8) - (5/8)\log_2(5/8) = 0.531 + 0.424 = 0.954$$

### Hair (Blond, Dark, Red)

Blond 2P 2N  $\rightarrow 1.0$  ; Dark 0P 3N  $\rightarrow 0$  ; Red 1P 0N  $\rightarrow 0$

$$\text{Weighted} = (4/8) \times 1.0 + (3/8) \times 0 + (1/8) \times 0 = 0.5$$

$$\text{Gain(Hair)} = 0.954 - 0.5 = 0.454$$

### Eyes and Height

Eyes: weighted 0.951  $\rightarrow$  Gain = 0.003 Height: weighted 0.951  $\rightarrow$  Gain = 0.003

### Pick the root

Hair has the highest Gain (0.454), so split on Hair first.

**Exam tip:** ID3 uses Entropy and Information Gain; CART uses the Gini index and makes binary trees. Both pick the split that makes the child groups as pure as possible.

## 8. Accuracy — Confusion Matrix and Metrics

After training a classifier we measure how good it is. Accuracy alone can mislead, so we also use the confusion matrix, precision, recall and the F1 score.

### The confusion matrix

|            | Predicted Yes     | Predicted No      |
|------------|-------------------|-------------------|
| Actual Yes | TP true positive  | FN false negative |
| Actual No  | FP false positive | TN true negative  |

- **TP** actual yes, predicted yes (correct). **TN** actual no, predicted no (correct).
- **FP** actual no, predicted yes (false alarm). **FN** actual yes, predicted no (a miss).

### The four metrics

$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$

$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

$\text{F1} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- Use **precision** when a false alarm is costly (flagging a good transaction as fraud).
- Use **recall** when a miss is costly (failing to detect a disease).
- **F1** balances precision and recall in one number.

#### ■ Numerical — from a list of 10 predictions

Rows of actual | predicted:

T | T T | F F | T F | F F | T T | T T | T T | F F | T T | T

##### Step 1 — count the four cells

TP = 4 FN = 2 FP = 3 TN = 1 (4+2+3+1 = 10)

##### Step 2 — apply each formula

**Formula** →  $\text{Accuracy} = (\text{TP} + \text{TN}) / \text{total}$

$\text{Accuracy} = (4 + 1) / 10 = 0.5$

**Formula** →  $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

$\text{Precision} = 4 / (4 + 3) = 4/7 = 0.571$

**Formula** →  $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

$\text{Recall} = 4 / (4 + 2) = 4/6 = 0.667$

**Formula** →  $\text{F1} = (2 \times P \times R) / (P + R)$

$\text{F1} = (2 \times 4/7 \times 4/6) / (4/7 + 4/6) = 8/13 = 0.615$

#### ■ Why accuracy alone can mislead

9990 records of class 0 and only 10 of class 1.

A model that always predicts class 0 scores 9990/10000 = 99.9% accuracy, yet finds no class 1 case at all.

So for imbalanced data, use precision and recall, not just accuracy.

*End of notes. Good luck in the exam.*